TradeShield

Database Schema & Entity Relationship Design

Project purpose

TradeShield is a mini stock-market simulator with an integrated fraud/anomaly monitoring engine. Users and simulated trading bots place orders; the trading engine validates and processes them; the Shield analyses order activity and records risk alerts. Market-price data and simulated order activity are deliberately kept separate.

Recommended database: SQLite for a compact student project. The schema below is designed for a working prototype and can later be migrated to PostgreSQL.

Key design rule: bots and manual users must pass through the same order-processing and fraud-detection pipeline. Do not create a separate shortcut path for bot orders.

1. Entity relationship overview

Parent entity	Relationship	Child entity	Meaning
TRADERS	1 → many	ORDERS	A trader can place many orders.
STOCKS	1 → many	ORDERS	Each order refers to one stock.
ORDERS	1 → zero/many	TRADES	An order may execute as one or more trades.
TRADERS	1 → many	POSITIONS	A trader can hold positions in multiple stocks.
STOCKS	1 → many	POSITIONS	Each position belongs to one stock.
TRADERS	1 → many	ALERTS	A trader can accumulate multiple risk alerts.
ORDERS	1 → zero/many	ALERTS	An alert may refer to a specific order; trader-level alerts may have no order ID.
TRADERS	1 → 1	PORTFOLIOS	One portfolio summary per trader (optional if balances/positions are computed).

2. Table definitions

TRADERS — people and simulated bots that can place orders

Column	Type	Rules	Purpose
trader_id	INTEGER	PRIMARY KEY	Unique trader identifier.

Column	Type	Rules	Purpose
name	TEXT	NOT NULL	Display name.
trader_type	TEXT	NOT NULL; HUMAN or BOT	Distinguishes a user from a simulated trader.
bot_strategy	TEXT	NULL for humans	For example NORMAL, HIGH_VOLUME, RAPID_CANCEL.
cash_balance_paise	INTEGER	NOT NULL; DEFAULT 0; CHECK >= 0	Cash stored as integer paise to avoid floating-point money errors.
created_at	TEXT	NOT NULL; ISO-8601	Creation timestamp.

STOCKS — instruments available in the simulator

Column	Type	Rules	Purpose
stock_id	INTEGER	PRIMARY KEY	Unique stock identifier.
symbol	TEXT	NOT NULL; UNIQUE	Ticker such as RELIANCE, TCS or INFY.
company_name	TEXT	NOT NULL	Company display name.
exchange	TEXT	NOT NULL; e.g. NSE	Exchange label for the selected price source.
current_price_paise	INTEGER	NOT NULL; CHECK > 0	Latest displayed market price in paise.
updated_at	TEXT	NOT NULL; ISO-8601	When the displayed price was last refreshed.

2. Table definitions (continued)

ORDERS — all manual and bot buy/sell requests

Column	Type	Rules	Purpose
order_id	INTEGER	PRIMARY KEY	Unique order identifier.
trader_id	INTEGER	NOT NULL; FK → TRADERS	Who placed the order.
stock_id	INTEGER	NOT NULL; FK → STOCKS	Instrument being traded.
side	TEXT	NOT NULL; BUY or SELL	Order direction.
quantity	INTEGER	NOT NULL; CHECK > 0	Number of shares requested.
limit_price_paise	INTEGER	NOT NULL; CHECK > 0	Maximum buy price or minimum sell price.
status	TEXT	NOT NULL; PENDING / PARTIALLY_FILLED / EXECUTED / CANCELLED / REJECTED	Current order state.
created_at	TEXT	NOT NULL; ISO-8601	Order submission time.
updated_at	TEXT	NOT NULL; ISO-8601	Most recent state change.

TRADES — successful executions

Column	Type	Rules	Purpose
trade_id	INTEGER	PRIMARY KEY	Unique execution identifier.
order_id	INTEGER	NOT NULL; FK → ORDERS	Order that generated this execution.
executed_quantity	INTEGER	NOT NULL; CHECK > 0	Quantity filled in this execution.
executed_price_paise	INTEGER	NOT NULL; CHECK > 0	Actual execution price in paise.
executed_at	TEXT	NOT NULL; ISO-8601	Execution timestamp.

POSITIONS — current holdings per trader and stock

Column	Type	Rules	Purpose
position_id	INTEGER	PRIMARY KEY	Unique position identifier.
trader_id	INTEGER	NOT NULL; FK → TRADERS	Owner of the holding.

Column	Type	Rules	Purpose
stock_id	INTEGER	NOT NULL; FK → STOCKS	Stock held.
quantity	INTEGER	NOT NULL; CHECK >= 0	Current number of shares held.
average_buy_price_paise	INTEGER	NOT NULL; CHECK >= 0	Average acquisition cost per share.
UNIQUE(trader_id, stock_id)	—	UNIQUE constraint	Only one aggregated position per trader/stock pair.

2. Table definitions (continued)

ALERTS — fraud/anomaly findings produced by Shield

Column	Type	Rules	Purpose
alert_id	INTEGER	PRIMARY KEY	Unique alert identifier.
trader_id	INTEGER	NOT NULL; FK → TRADERS	Trader whose activity is being assessed.
order_id	INTEGER	NULL; FK → ORDERS	Optional link to a triggering order.
rule_name	TEXT	NOT NULL	E.g. HIGH_VOLUME, REPEATED_CANCELLATION, HIGH_FREQUENCY.
severity	TEXT	NOT NULL; LOW / MEDIUM / HIGH	Human-readable severity.
risk_points	INTEGER	NOT NULL; CHECK 0–100	Illustrative contribution to the demo risk score.
description	TEXT	NOT NULL	Explain what pattern triggered the alert.
created_at	TEXT	NOT NULL; ISO-8601	When the alert was generated.
review_status	TEXT	NOT NULL; DEFAULT OPEN	OPEN / REVIEWED / CLEARED.

PORTFOLIOS — optional cached summary per trader

Column	Type	Rules	Purpose
portfolio_id	INTEGER	PRIMARY KEY	Unique portfolio identifier.
trader_id	INTEGER	NOT NULL; UNIQUE; FK → TRADERS	One portfolio per trader.
total_value_paise	INTEGER	NOT NULL; DEFAULT 0	Cached portfolio value; can also be calculated from cash and positions.
updated_at	TEXT	NOT NULL; ISO-8601	Last summary refresh.

3. Important constraints and indexes

Enable SQLite foreign-key enforcement on every database connection: `PRAGMA foreign_keys = ON;`

Rule / index	Reason
CHECK quantity > 0 on ORDERS and TRADES	Prevents zero or negative share quantities.
CHECK prices > 0	Prevents invalid order or execution prices.

Rule / index	Reason
UNIQUE(symbol) on STOCKS	Prevents duplicate ticker records.
UNIQUE(trader_id, stock_id) on POSITIONS	Keeps one aggregate holding row per trader and stock.
INDEX ORDERS(trader_id, created_at)	Speeds up trader activity and frequency checks.
INDEX ORDERS(stock_id, created_at)	Speeds up stock-specific order history.
INDEX ALERTS(trader_id, created_at)	Speeds up Shield Monitor alert history.
INDEX ORDERS(status, created_at)	Helps retrieve pending and recent orders.

4. Order-processing and detection flow

1. Manual user or bot creates an order → **2.** validate trader, stock, quantity, price, balance and holdings → **3.** save order → **4.** send the order event to FraudEngine → **5.** run detection rules over a time window → **6.** save any alerts → **7.** execute or update order status → **8.** update cash/positions and refresh the dashboard.

Detection rules to demonstrate: unusual order volume, repeated cancellations, and unusually high order frequency. A large order by itself should not be labelled fraud; it should contribute to a risk score and be interpreted with other activity.

5. Suggested API-to-database mapping

API endpoint	Database tables	Purpose
GET /api/stocks	STOCKS	List instruments and latest displayed prices.
GET /api/portfolio	PORTFOLIOS, POSITIONS, STOCKS, TRADERS	Return cash, holdings and portfolio summary.
GET /api/orders	ORDERS, STOCKS, TRADERS	Show manual and simulated order history.
POST /api/orders	TRADERS, STOCKS, ORDERS, ALERTS	Validate, store and analyse a new order.
POST /api/orders/:id/cancel	ORDERS, ALERTS	Cancel an eligible order and analyse cancellation behaviour.
GET /api/traders	TRADERS	List human and bot traders.
GET /api/alerts	ALERTS, TRADERS, ORDERS	Display Shield alerts.
GET /api/risk/:traderId	ALERTS, ORDERS	Calculate or return a trader risk summary.
POST /api/simulation/start	TRADERS, ORDERS	Start bot-generated order activity.
POST /api/simulation/stop	—	Stop generation of new bot orders.
GET /api/simulation/status	—	Return current simulator state.

6. Implementation notes

- Use integer paise for all stored monetary values (■1 = 100 paise); convert to rupees only for display.
- Keep market-price feed data separate from simulated order/trade records. A quote API supplies prices; it does not provide your demo's bot order flow.
- Treat this as a classroom simulator, not a real brokerage or exchange. Do not connect it to live order execution.
- Keep fraud detection explainable: store the rule name, risk points and plain-language reason for each alert.
- For a short project, PORTFOLIOS can be omitted and calculated from TRADERS cash plus POSITIONS; retain it only if you want cached dashboard summaries.
- Use database transactions when updating an order, trade, cash balance and position so partial updates cannot leave inconsistent data.